Aarhus university

Computer-Science

Distributed Systems and Security

Handin 7

Author	*Student number*
Søren M. Damsgaard	**202309814**
Alexander	**202408929**
Thor behrmann	**202410271**

April 9, 2026

Implementation of signed transactions

The Signatures are implemented, placing the data [TxID, FromAccount, ToAccount, Amount] into a separate struct that is serialized with json, which can be used to create the signature with the **RSASign** method from exercise 6.15.
The serialized data and the signature can then be placed inside a shared struct that acts as a signed transactions.

Encoding of RSA as identifiers for accounts

The encoding of the RSA public keys, was done by having a PublicKey struct containing [N, E] as a single element, then the struct can be serialized and converted to string, that represent an account, this means whenever someone needs an accounts public key the account name can be easily de-hashed to access E and N.

Why are the user accounts secure?

Even though this isn't proper encryption like AES or RSA, an RSA Signature in this case is still provides a security layeren via authentication.
It is know that RSA is mathematically secure, and for an advesary to forge a transactions they need to forge the Signature or else it will be ignore by the protocol, and to forge the Signature they would need to crack the RSA private key d, which isn't possible atm. hence the user accounts are quite secure.

Testing [Not applicable]

Automatic testing wasn't implemented in this program.

How to run code

When code is unzipped, go into the terminal and make sure you are in the directory below

```
    .\Handin_7
```

and to run the code use the following commands in the terminal

```
    go build .
    go run .
```

this builds and runs all necessary files, including MAIN.GO

Backlog

- Nothing crucial since Automatic testing isn't planned until exercise 16.2

AI usage

AI was not used.